

Topic 1.5

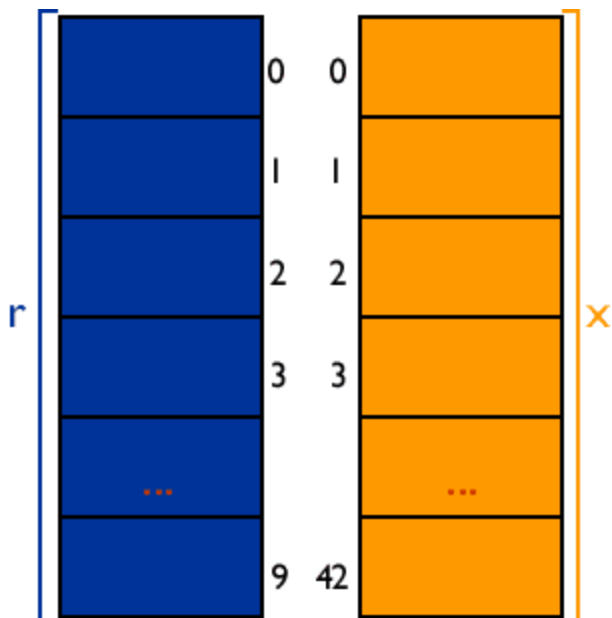
Topic 1.5: Arrays in C

An *array* is a collection of data values. An element of the collection is referred to by the name of the collection (the array name), and the element's position within the collection (the index).

The syntax for declaring arrays is:

```
int r[10];  
double x[43];
```

which declare *r* as an array of 10 *ints*, and *x* as an array of 43 *doubles*.



All C arrays use 0-based indexing. Thus:

```
r[0]
```

refers to the first element of *r*. To refer to the *i*th element of an array, use the notation:

```
r[i]
```

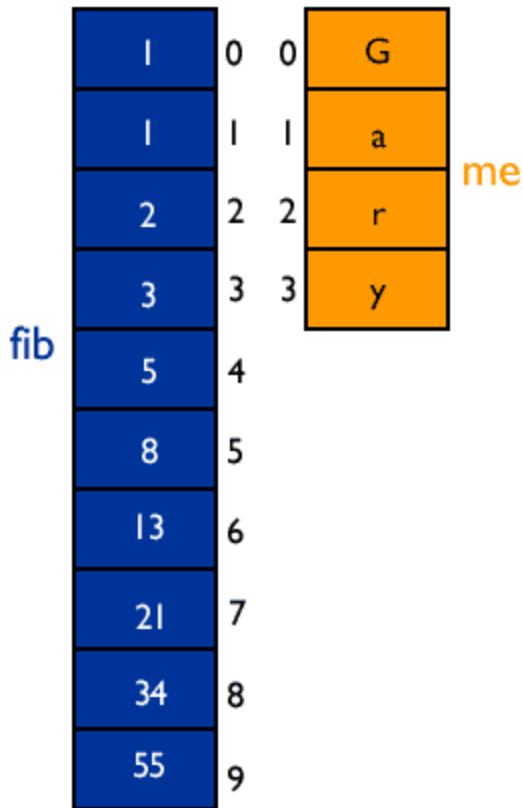
where *i* is any expression producing an *int* as a result.

Out-of-range subscripts cause references and overwrites to memory locations not within the array, resulting in errors which may be extremely difficult to find.

We usually have 3 pieces of information when dealing with an array: the array itself, the actual size (capacity) of the array, and the number of elements currently in use. Normally, when we fill an array we start at subscript 0 and move toward higher subscripts.

An array may be initialized in its declaration:

```
int  fib[10] = {1,1,2,3,5,8,13,21,34,55};
char me[4]  = {'G','a','r','y'};
```



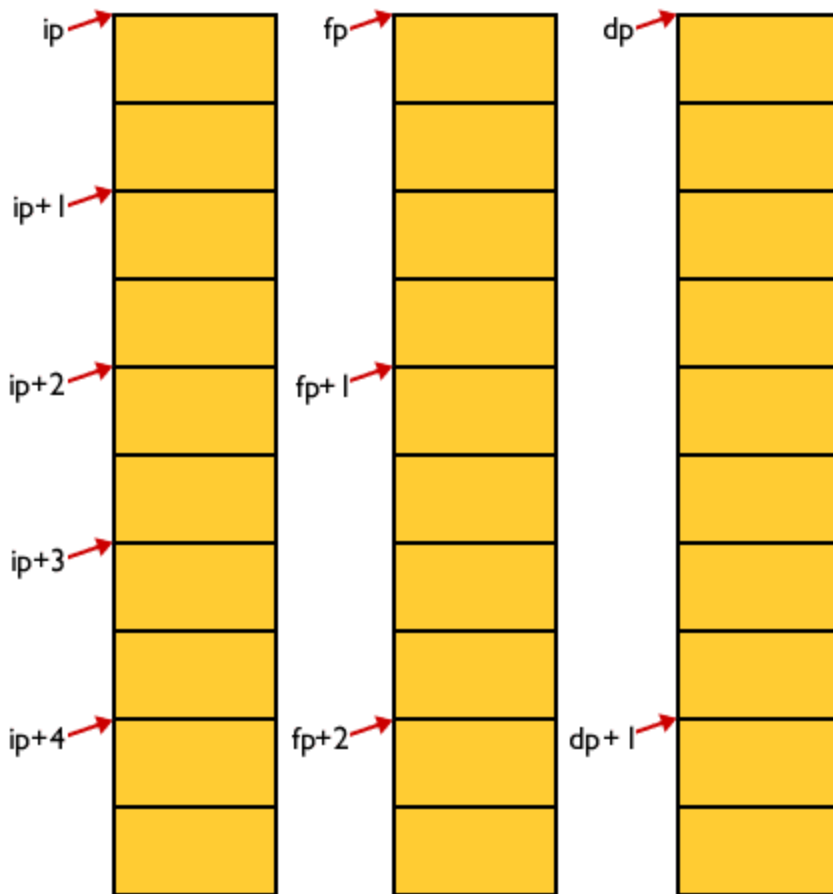
Pointers and Arrays

The *value* of an array is *the address of its first element*. That is, if you simply use the name `r` by itself, it means the same as `&r[0]`. *This is the most important fact in the entire C language, because it explains why many things work.*

If `p` is a pointer to an `int`, adding one to `p` makes it point to the next `int` in memory. The same holds for doubles and other types. In the following diagram, assume an `int` occupies 2 bytes, a `float` occupies 4 bytes, and a `double` occupies 8 bytes:

```
int    *ip;
float  *fp;
double *dp;
```

This diagram shows what we have after initializing the above pointers to point to well defined places:



As a consequence of these rules:

```
int x[12];

int *p = x; /* Create a pointer that points at
             first element of x. */
```

p and x now mean the same thing (for as long as p points to $x[0]$):

$x[i]$ refers to an element of x .

$*(p + i)$ refers to the same element of x .

$*(x + i)$ refers to the same element of x .

$x[i]$ and $p[i]$ always mean the same thing.

to topic 1.6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-1-dot-6>)